

Investigating Scale Independent UCT Exploration Factor Strategies

Robin Schmöcker - *Leibniz Universität Hannover* - schmoecker@tnt.uni-hannover.de

Christoph Schnell - *Leibniz Universität Hannover* - schnell@tnt.uni-hannover.de

Alexander Dockhorn - *University of Southern Denmark* - adoc@mmmi.sdu.dk

Abstract—The Upper Confidence Bounds For Trees (UCT) algorithm is not agnostic to the reward scale of the game it is applied to. For zero-sum games with the sparse rewards of $\{-1, 0, 1\}$ at the end of the game, this is not a problem, but many games often feature dense rewards with hand-picked reward scales, causing a node’s Q-value to span different magnitudes across different games. In this paper, we evaluate various strategies for adaptively choosing the UCT exploration constant λ , called λ -strategies, that are agnostic to the game’s reward scale. These λ -strategies include those proposed in the literature as well as five new strategies. Given our experimental results, we recommend using one of our newly suggested λ -strategies, which is to choose λ as $2 \cdot \sigma$ where σ is the empirical standard deviation of all state-action pairs’ Q-values of the search tree. This method outperforms existing λ -strategies across a wide range of tasks both in terms of a single parameter value and the peak performances obtained by optimizing all available parameters.

Index Terms—Games, Artificial Intelligence, MCTS, UCT, Sequential Decision-making.

I. INTRODUCTION

Monte Carlo Tree Search (MCTS) [7], [8] is a popular model-based decision-making algorithm that is applicable to a wide range of games. A selection strategy within MCTS that is nowadays synonymously used with MCTS is Upper Confidence Bounds For Trees (UCT) [16], which balances exploration and exploitation by employing the Upper Confidence Bounds (UCB) algorithm for multi-armed bandits as the tree policy [16].

UCT, however, introduces one parameter, the exploration constant $\lambda > 0$ used for the UCB formula whose choice can significantly impact performance (see Section IV). While a single value for λ may perform well across a variety of games with a similar scale for the Q-values, if λ is not on the same scale as the node’s Q-value, UCT may either degrade to uniformly picking actions during the tree policy (when $\lambda \gg |Q|$) or to the greedy policy (when $\lambda \ll |Q|$). This is problematic as the scale of the score which the agent tries to maximize is oftentimes arbitrarily chosen. The games that are later used for evaluation prove this point as they encompass scores ranging from one to four digits.

While several works discuss methods for choosing a λ that adapts to the problem’s scale [2], [6], [5], [11], [22] these works, however, most treat this choice as an implementation detail.

In this paper, we evaluate the performance of several of these strategies as well as propose new strategies for adaptively choosing λ on a variety of single-player and two-player games and show that the methods suggested in the literature are

outperformed by the methods introduced here. From here, we refer to any method for choosing λ as a λ -strategy. Since the goal of this paper is to recommend a drop-in λ -strategy replacement for Vanilla UCT (i.e., using a fixed λ value), the method had to fulfill four criteria, which are all satisfied by our newly proposed **Global Std** strategy that chooses λ as $C \cdot \sigma$ where $C \in \mathbb{R}^+$ and σ is the empirical standard deviation of all state-action pairs’ Q-values.

- 1) Scale independence: If the reward function of a problem is multiplied by a constant $\mu > 0$, then this must not affect the policy induced by UCT.
- 2) The strategy must have little to no computational overhead.
- 3) There should be a single parameter of the λ -strategy that outperforms any single Vanilla UCT exploration constant choice across games. For Global Std, this is the case for $C = 2$.
- 4) Optimizing over the parameters of the λ -strategy (if any parameters are introduced) on a per-task-level should outperform UCT when its exploration constant is also optimized.

Our contributions can be summarized as follows.

- We conducted a large-scale study on several λ -strategies from the literature as well as newly proposed ones by evaluating them on a plethora of single-player and two-player decision-making tasks with varying reward scales.
- One strategy that we proposed, namely Global Std, outperforms all methods from the literature both in terms of generalization and peak performance, whilst simultaneously satisfying the above-mentioned criteria.
- In particular, we show that Vanilla UCT, i.e., using a fixed exploration constant, performs poorly across the problems considered here that have vastly different reward scales.

The rest of this paper is structured as follows. Firstly, we present related work in **Section II**. Then, in **Section III**, we introduce three λ -strategies from the literature and propose five new ones, including the above-mentioned Global Std strategy. Given the theoretical foundations, in **Section IV**, we describe our experiment setup and then evaluate UCT with the different λ -strategies on six problems. Lastly, in **Section V**, we briefly summarize our findings and give an outlook for future work.

II. RELATED WORK

Surprisingly, there is very little research conducted on dynamically choosing the exploration constant λ in MCTS (or equivalently normalizing Q-values, see Section III) [7]

on-the-fly. Even less research has been conducted on scale-invariant exploration strategies. We suspect this to be the case since this is only a minor detail in MCTS, mostly not worthy of spending dedicated research effort on. Further, we also assume that there are numerous papers we are not aware of that employed some adaptive λ -strategy for their UCT. We assume this because authors may not have advertised their λ -strategy as it was only considered a minor implementation detail.

Tomáš Kozelek [17] suggested two λ -strategies, one where λ decays proportional to $\frac{1}{\sqrt{n}}$ and one where λ is proportional to the empirical variance of the current node's returns. In both strategies, λ is clipped within a fixed range. Both Auer et al. [4] and later Audibert et al. [3] also proposed reward-variance-based λ -strategies called UCB-tuned and β -UCB. For each action, they use the variance of the state-action pair's returns instead of the entire node's returns. Yet another λ -strategy paradigm was introduced by Moerland et al. [19] who used a λ value that directly reflects the size of the current node's subtree, which quantifies uncertainty, thus improving exploration. Even without clipping, none of these strategies are independent of the scale of the Q-values.

One of the few documented scale-independent tuning approaches is choosing $\lambda = Q_a$ [5], [6], [11] where Q_a is the Q-value of the action a under consideration at this node. However, a big flaw of this approach is that any actions with negative Q-values are always ignored no matter the visit count, when there is at least one action with a positive Q-value. This issue can be avoided by using $\lambda = |Q_a|$ as done by Anand et al in OGA-UCT [2]. However, this comes with the problem that there is a bias towards actions with $Q_a \ll 0$. This difficulty can be circumvented with a λ -strategy proposed by Schrittwieser et al. [22] who choose $\lambda = Q_{\max}^T - Q_{\min}^T$ where $Q_{\max}^T, Q_{\min}^T$ are the highest and lowest Q values of the current search tree T . Yet another scale-independent approach that is also reward-variance-based is given by Gray et al.'s [13] strategy called *Poly-UCB1 with derived C* that considers different confidence levels of the Student's t-distribution. In contrast to our Global Std method, Gray et al. build the variance from the returns of a single Q-value, while Global Std uses the variance of the different Q values.

In a broader scope, there exists a plethora of techniques to improve MCTS [7], some of which directly target the tree policy, such as work by Galván et al. [12], who use evolutionary algorithms to find alternatives for the entire UCB formula or techniques that only partially affect the exploration term, such as work by Sironi et al. [24] who propose and review several online parameter tuning techniques that interleave UCT iterations and parameter selection. Aside from MCTS, parameters in a more general reinforcement learning setting can also be scale-dependent, such as the temperature in Soft Actor Critic, for which automatic tuning methods exist [10].

III. METHOD

A. Problem model and optimization objective

Firstly, we use finite Stochastic Games (SG) as the model for the environments/tasks/problems considered here, which

UCT formally operates on. This subsequent definition is a slight modification of Shapley's original definition [23]. In the following, $\Delta(X) \subseteq \mathbb{R}^n$ denotes the probability simplex for any non-empty set X with $|X| = n$.

Definition III.1. A SG is an 8-tuple $M = (S, n_p, \mu_0, T, I, \mathbb{A}, \mathbb{P}_t, R)$ where the components are as follows:

- $S \neq \emptyset$ is the finite set of states, $n_p \in \mathbb{N}$ is the number of players, $\mu_0 \in \Delta(S)$ is the probability distribution for the initial state, and $T \subseteq S$ is the (possibly empty) set of terminal states.
- $I: (S \setminus T) \mapsto \{1, \dots, n_p\}$ maps each non-terminal state to which of the n_p players is at turn.
- $\mathbb{A}: (S \setminus T) \mapsto \mathcal{A}$ maps each state s to the available actions $\emptyset \neq \mathbb{A}(s) \subseteq \mathcal{A}$ at state s where $|\mathcal{A}| < \infty$. Again, we denote the set of permissible state-action pairs by $P_M = \{(s, a) \mid s \in (S \setminus T), a \in \mathbb{A}(s)\}$.
- $\mathbb{P}_t: P_M \mapsto \Delta(S)$ is the stochastic transition function where we use $\mathbb{P}_t(s' \mid s, a)$ to denote the probability of transitioning from $s \in S$ to $s' \in S$ after taking action $a \in \mathbb{A}(s)$ in s .
- $R: P_M \mapsto \mathbb{R}^{n_p}$ is the reward function that outputs the reward of each player.

In particular, note that we assume that the immediate reward deterministically depends on the current state and chosen action only, and that each state has exactly one player who may choose the next action. Furthermore, if we choose $n_p = 1$, then one obtains an MDP [25].

From hereon, let $M = (S, \mu_0, \mathbb{A}, \mathbb{P}, R, T)$ be a SG. The goal is to find an agent $\pi: S \mapsto \Delta(\mathcal{A})$ that is modelled as a mapping from states to action distributions such that π maximizes the expected episode's return for player $1 \leq i \leq n_p$ where the (discounted) return for of episode $s_0, a_0, r_0, \dots, s_n, a_n, r_n, s_{n+1}$ with $s_{n+1} \in T$ is given by $\gamma^0(r_0)_i + \dots + \gamma^n(r_n)_i$.

B. MCTS foundation and equivalences between normalization and λ -strategies

The tree policy in UCT for fully expanded nodes is given by selecting the action with the highest UCB value, with random tie-breaking at each node. The UCB value at a node $\mathcal{N}$ of an action $a \in \mathcal{A}$ with $N_a \in \mathbb{N}$ visits and an estimated Q-value $Q_a \in \mathbb{R}$ (i.e. accumulated reward divided by the number of visits) with $\mathcal{N}$ having $n \in \mathbb{N}$ total visits is given by $Q_a + \lambda \cdot \sqrt{\frac{\log n}{N_a}}$, where $\lambda \in \mathbb{R}^+$ is the exploration constant. In this paper, we only discuss strategies for picking λ , however, this is equivalent to any Q-value normalizing strategies, since it holds that

$$\operatorname{argmax}_{a \in \mathcal{A}} Q_a + \lambda \cdot \sqrt{\frac{\log n}{N_a}} = \quad (1)$$

$$\operatorname{argmax}_{a \in \mathcal{A}} \frac{Q_a - Q_{\text{lower}}}{Q_{\text{upper}} - Q_{\text{lower}}} + \sqrt{\frac{\log n}{N_a}} \quad (2)$$

with $Q_{\text{lower}}, Q_{\text{upper}} \in \mathbb{R}$ being some normalizing constants, $0 \neq \lambda = Q_{\text{upper}} - Q_{\text{lower}}$.

C. Introduction of λ -strategies

Instead of λ being constant, we treat it as a function of the current search tree $\mathcal{T}$, the tree policy decision node $\mathcal{N}$, and the considered action a to allow for information flow between all nodes and not just those on the same path to the root.

Next, we denote the different variants for $\lambda := \lambda(\mathcal{T}, \mathcal{N}, a)$ that we considered where we assume λ to have the form $\lambda = C \cdot \hat{\lambda}$, $C \in \mathbb{R}$. Even though this form still contains a parameter, this one will be by construction scale independent and less sensitive to the specific environment as we shall see later. After we introduce the $\hat{\lambda}$ variants, we will give additional reasoning for why this C is necessary. The variants with the names {Global Abs, **Global Std**, Layer Range, Layer Abs, Layer Std, Local Range, Local Std} are introduced by us, while the others have already been used throughout the literature.

Vanilla UCT: $\hat{\lambda} = \hat{\lambda}_0$ is constant. This strategy was originally proposed by Kocsis and Szepesvári [16] and does not perform any dynamic adjustment of $\hat{\lambda}$.

Global Range: $\hat{\lambda} = Q_{max}^{\mathcal{T}} - Q_{min}^{\mathcal{T}}$ where $Q_{max}^{\mathcal{T}}, Q_{min}^{\mathcal{T}}$ denote the maximum and minimum Q-value of all state-action pairs in $\mathcal{T}$. This strategy was proposed by Schrittwieser et al. [22] and does arguably not satisfy the no-computational-overhead requirement as one either has to regularly iterate the entire tree or rearrange a sorted list of all Q values to know the min and max Q value. However, we still included it for comparison purposes and as we shall see later it performs worse than our Global Std method.

Global Abs: $\hat{\lambda}$ is set to the average absolute Q-value of all state-action pairs in τ .

Global Std: $\hat{\lambda} = \sigma^{\mathcal{T}}$, where $\sigma^{\mathcal{T}}$ denotes the empirical standard deviation of the Q-values of all state-action pairs in $\mathcal{T}$. Hence, if $\mathcal{T}$ contains n state-action pairs, $\sigma^{\mathcal{T}}$ is the standard deviation of n data points.

Layer Range: $\hat{\lambda} = Q_{max}^l - Q_{min}^l$ where Q_{max}^l, Q_{min}^l denote the maximum and minimum Q-value of all state-action pairs in the same layer as $\mathcal{N}_a$. As with Global Range, this method also brings a computational overhead with it, however, as we shall see later, Global Std outperforms it nonetheless.

Layer Abs: $\hat{\lambda}$ is set to the average absolute value of all state-action pairs in the same layer as $\mathcal{N}$.

Layer Std: $\hat{\lambda} = \sigma^l$, where σ^l denotes the empirical standard deviation of the Q-values of all state-action pairs in the same layer as $\mathcal{N}_a$.

Local Range: $\hat{\lambda} = Q_{max}^{\mathcal{N}} - Q_{min}^{\mathcal{N}}$ where $Q_{max}^{\mathcal{N}}, Q_{min}^{\mathcal{N}}$ denote the maximum and minimum Q-value of all state-action pairs of $\mathcal{N}$.

Local Std: $\hat{\lambda} = \sigma^{\mathcal{N}}$, where $\sigma^{\mathcal{N}}$ denotes the empirical standard deviation of the Q-values of all state-action pairs of $\mathcal{N}$.

Local Abs Q: $\hat{\lambda} = |Q_a|$ where Q_a denotes the Q-value of action a at $\mathcal{N}$. This strategy was used by Anand et al. [2].

Local Q: $\hat{\lambda} = Q_a$ where Q_a denotes the Q-value of action a at $\mathcal{N}$. This strategy is the most common in literature [5], [6], [11].

Poly-UCB1: $\hat{\lambda} = \frac{\sigma_a \cdot \varphi(N_a)}{\log n}$ where n is the number of visits of node $\mathcal{N}$, N_a is the number of times action a has been visited at node $\mathcal{N}$, $\varphi(N_a)$ is the 99% quantile of the Student's t-distribution with $N_a - 1$ degrees of freedom, and σ_a is the empirical standard deviation for the state-action pair $(\mathcal{N}, a)$. Note that σ_a is not the same as $\sigma^{\mathcal{N}}$ as the latter considers different Q values, while σ_a considers individual returns of a single Q-value.

D. Theoretical discussion of the λ -strategies

Why the parameter C is necessary: Even though the above-mentioned λ -strategies are reward scale independent, some problems may favor more explorative tree-policies while other favor more exploitative ones. Hence, we still need the parameter C to control this.

Also note that since the different scopes (e.g. global, layer, local) form a hierarchy of nodes, it follows that $Q_{max}^{\mathcal{T}} - Q_{min}^{\mathcal{T}} \geq Q_{max}^l - Q_{min}^l \geq Q_{max}^{\mathcal{N}} - Q_{min}^{\mathcal{N}}$. Even though all of these compute the same quantity in principle, their scale differs which can be fixed by the multiplication with C depending on the strategy used.

Fulfilment of scale independence: Observe that these λ -strategies satisfy the scale independence condition posed in the introduction as all presented variants for λ use functions f (namely: $|\cdot|$, σ , $\max - \min$, id) with the homogeneity property that for any $\mu > 0$ and input x it holds that $f(\mu x) = \mu f(x)$.

The validity of achieving reward scale independence by choosing homogenous function is supported by reconsidering the convergence analysis of the original UCT paper [16] where Kocsis and Szepesvári assumed all cumulative rewards to lie in the interval $[0, 1]$. If one were to preserve their results for arbitrary bounds $[a, b]$, one simply needs to scale the exploration constant with $b - a$. This is because their analysis builds on bounds obtained by Hoeffding's inequality [15] which states for any independent random variables $a \leq X_1, \dots, X_n \leq b$ and any $t > 0$ it holds that

$$\mathbb{P} \left(\sum_{i=1}^n (X_i - \mathbb{E}[X_i]) \geq t \right) \leq e^{-\frac{2nt^2}{(b-a)^2}}. \quad (3)$$

Without further specifying what the X_i are, Kocsis and Szepesvári insert the exploration constant for t . Hence, to obtain the same bounds as for $(a, b) = (0, 1)$ one has to replace t (and subsequently the exploration constant) with $(b - a)t$.

Fulfilment of low computational overhead: The λ -strategies introduced here, except for the range strategies, also fulfill the remaining two conditions posed in the introduction. For calculating the standard deviation, one only needs to keep track of the number of Q-values in consideration, their sum, and their squared sum and then make use of the identity $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$ for a random variable X . The bookkeeping needed for all strategies can be computed during the backpropagation phase of MCTS: The old Q- and squared Q-value is subtracted from the statistics under consideration and the updated ones are added back.

Additive invariance of std and range: Lastly, note that the std and range strategies do not change their output when a constant is added to all Q-values their computation is based on.

This property is desirable for MDPs with fixed episode lengths as the optimal policy in these cases is invariant to adding a constant to the reward function.

Convergence of std strategies: While both the std and range strategies fulfill all of the above-mentioned properties, range strategies are not guaranteed to converge. Imagine an MCTS search tree where the Q-values are sampled from a Gaussian. While the std strategies will eventually converge to a unique value, all range strategies will diverge to positive infinity with an increase in the search tree size as more outliers are sampled. In general, the range strategies are vulnerable to outlier values.

Localities: Most methods can be grouped by their locality, i.e. global, versus layerwise, versus local. Each comes with a different theoretical tradeoff: The larger the scope, the more values can be aggregated to compute the desired quantity (i.e., standard deviation, absolute value, or range) but the less accurate these estimates are for the current node $\mathcal{N}$ in question. Fig. 1 visualizes the different localities.

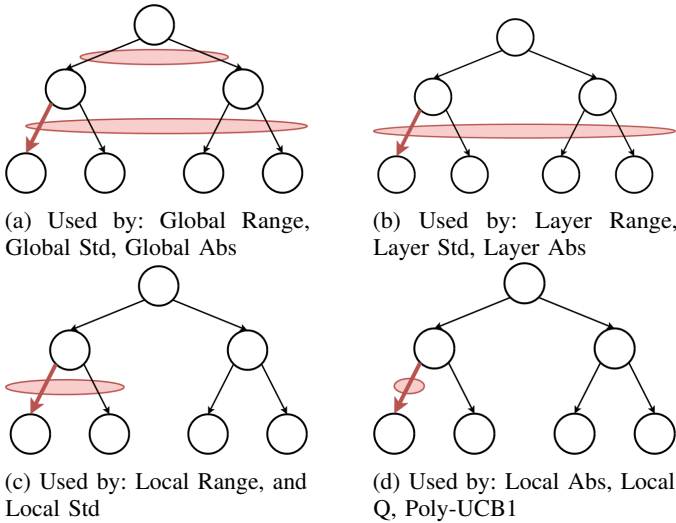


Fig. 1: A visualization of which state-action pairs are taken into consideration when determining the exploration factor λ for the bottom-left state-action pair that is marked in red. The four images all depict a search tree with five nodes where arrows represent deterministic actions. The state-action pairs used to determine the corresponding exploration constant all intersect a red ellipse. Each subfigure’s caption lists the λ -strategies that use the marked set of state-action pairs for the λ calculation.

IV. EXPERIMENTS

A. Experiment setup

In this section, we describe the experiment setup for the subsequent experiments.

Problem models: For this paper, we ran our experiments on 17 MDPs and 11 two-player games, all of which are either from the International Probabilistic Planning Conference [14], or are well-known board or strategy games. *List of MDPs:* Academic Advising, Cooperative Recon, Crossing Traffic, Earth Observation, Elevators, Game of Life, Manufacturer, Navigation, Push Your Luck, Racetrack, Red Finned Blue Eye,

Sailing Wind, Skill Teaching, Saving, SysAdmin, Tamarisk, Traffic, Triangle Tireworld, Wildfire, and Wildlife Preserve.

List of two-player games: Chess, Constrictor, Connect 4, Numbers Race, Othello, Quarto, Pylos, Capture the Flag, Kill the King, Pusher, and Tic Tac Toe.

In the supplementary materials (Section V-H), we provide a description of the above-mentioned problems. All experiments were run on the finite horizon versions of the considered MDPs with a default horizon of 50 steps and 200 for the two-player SGs with a planning horizon of 50 and a discount factor $\gamma = 1$. On SGs, agents were evaluated versus the fixed 500 iterations Global Std agent with $C = 4$. In particular, these tasks cover different reward scales, e.g., the two-player zero-sum games’ episode returns are either 1, -1, or 0, Game of Life’s episodes’ returns lie in the positive 3-digit region, while Sailing Wind’s returns are each within the negative 2-digit region.

Evaluation: Each data point that we denote in the remaining sections of this paper (e.g. agent returns) is the average of at least 2000 runs. For the two-player games, we ran 2000 games where the agent-to-be-evaluated has the first move and 2000 where the agent does not. Whenever we denote a confidence interval for a data point, then this is always a confidence interval with a confidence level of 99% provided by ≈ 2.33 times the standard error.

Normalized pairings score: Mostly, performance margins will be rather small. To be able to appropriately rank the λ -strategies without manually reviewing over 1000 individual performances, we will later construct so-called *normalized pairings-score matrices* and the *normalized pairings score* that is constructed as follows. Let $\{\pi_1, \dots, \pi_n\}$ be n agents (e.g., parameter-optimized λ -strategies, or individual λ -strategy-parameter pairs) where each agent was evaluated on m tasks (later, a task will be a given MCTS iteration budget and an environment). The corresponding normalized pairings score matrix is of size $n \times n$ and the entry (i, j) is equal to the number of tasks where π_i performed better than agent π_j subtracted by the number of times it performed worse, divided by m . The normalized pairings score s_i , $1 \leq i \leq n$ is given by averaging over the i -th row when excluding the i -th column.

Reproducibility: For reproducibility, we released our implementation which is available at <https://github.com/codebro634/DynamicExplorationFactorUCT>. Our code was compiled with g++ version 13.1.0 using the -O3 flag (i.e. aggressive optimization). For reproducibility purposes, we seeded all components of our program that require RNG. The RNG engine was reset for every Agent-Domain-Map triplet with the seed 42.

B. Experimental results

Using the previously described setup, we measured the mean episode return for each λ -strategy-environment pair using $n \in \{100, 500, 2500\}$ MCTS iterations and $C \in \{0.125, 0.25, 0.5, 1, 2, 4, 8, 16, 32, 64\}$ for all λ -strategies except for Vanilla UCT for which we used $C \in \{0.125, 0.25, 0.5, 1, 2, 4, 8, 16, 32, 64, 256, 1000\}$ to account for environments which high-magnitude reward scales. In the following, we will only be presenting a summary of these experiments. The full data table(s) can be found in the supplementary materials in Subsection V-I.

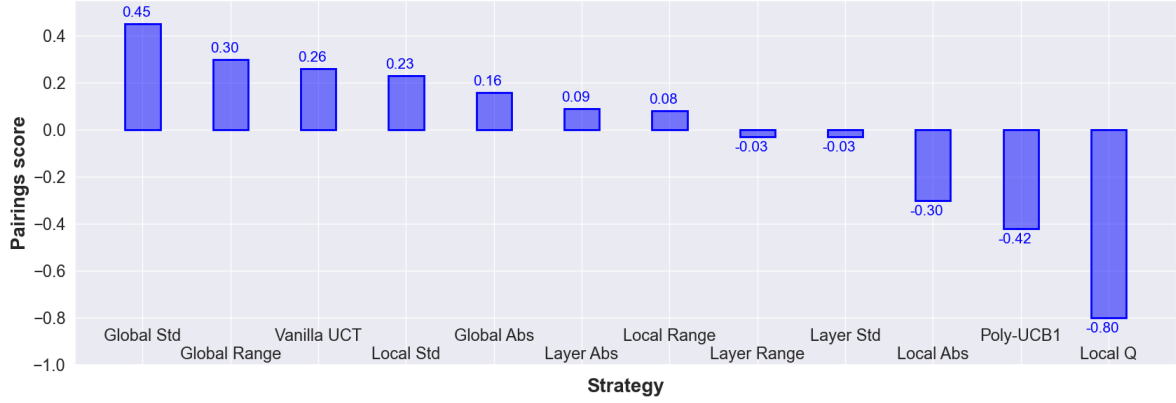


Fig. 2: The normalized pairings score for all considered λ -strategies. The score was constructed by considering all iteration budgets and all environments. Per pairing, the maximum performance over all C values (i.e. $\{0.125, 0.25, 0.5, 1, 2, 4, 8, 16, 32, 64\}$ for all strategies and additionally 256 and 1000 for Vanilla UCT) was used per iteration-environment pair. One of our methods, Global Std, performs best overall.

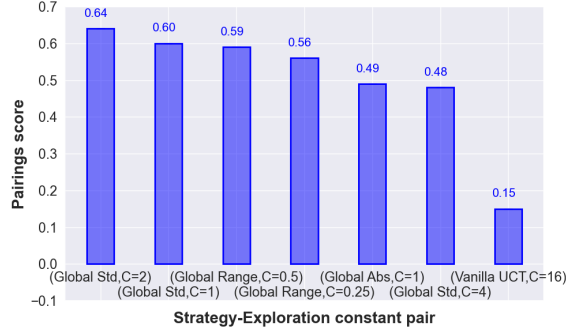


Fig. 3: The best 6 normalized pairings score for all considered λ -strategy and exploration constant C pairs as well the best UCT parameter. The score was constructed by considering all iteration budgets and all environments. One of our methods, Global Std with $C = 2$, performs best overall, while standard UCT is far worse than any of these methods, proving that scale-invariant methods are necessary. The score for all λ -strategy- C -pairs is found in the last column of Tab.VIII in the supplementary materials.

Investigating peak performances: First, we tested which of the strategies introduced here performs best when considering the best performing C value for each iteration budget-model pair. Bar chart 2 shows the normalized pairings score (see Section IV-A) for all 12 parameter-optimized λ -strategies across all iteration budgets. Firstly, our newly proposed strategy, Global Std, performed best overall, followed by a significant gap by Global range. This lead is kept for all the iteration budgets considered here. In the supplementary materials, we list the normalized pairing scores for each iteration budget in Tab.VII. Furthermore, in Tab. III, we denote the normalized pairings score matrix for all iteration budgets combined. Tables IV, V, and VI show the normalized pairings score matrices for the individual iteration budgets.

One can also make another key observation from this bar chart: Global strategies, that use all Q nodes as samples,

perform best overall, while the most local strategies, namely Local Q, Poly-UCB1, and Local Abs, perform worst overall.

Lastly, it has to be noted that while Global Std's pairings score far exceeds that of Vanilla UCT, this does not take the magnitude of performance improvements into account. Tab. I in the supplementary materials shows the exact performances for each individual environment of Global Std, Vanilla UCT, and the strategy that performed best in the corresponding environment. Expectedly, the relative performance improvement over Vanilla UCT is rather small (on average 4%), as optimizing Vanilla UCT per environment does not have any issues with varying reward scales.

Investigating generalization capabilities: Next, we investigate the generalization capabilities to determine which single parameter combination performs best overall. Bar chart 3 shows the top 6 normalized pairings scores for all λ -strategy, exploration constant C pairs as well as the best performing Vanilla UCT variant for comparison. Though with a smaller lead, our method Global Std performs best overall using the parameter $C = 2$. The second-best λ -strategy is Global Range with $C = 0.5$. Note that the best Vanilla UCT variant ($C = 16$) performs far worse than these scale-invariant methods, showing that fixed-parameter UCT can not deal with varying reward scales, as Vanilla UCT dropped from third place in the parameter-optimized setting to some middle-of-the-pack place in the generalization setting. To emphasize the reward scale sensitivity, Fig. 4 in the supplementary materials shows the performances of Global Std and Vanilla UCT with fixed C values but varying reward scales for the environments considered here. Again, the results are not dependent on the concrete MCTS iteration budget: The parameter $C = 2$ for Global Std performs best for all iteration budgets. Furthermore, except for the 100 setting where Global Std is nearly equal to Global range, Global Std always performs best. In the supplementary materials in Tab. VIII, we list the normalized pairings score for all iteration budgets.

Additionally, in contrast to the parameter-optimized setting, a single C value for Vanilla UCT is rarely adapted to the

problem’s reward scale. Tab. II in the supplementary materials reports per-environment results for Global Std ($C = 2$) and Vanilla UCT ($C = 16$). In this case, Global Std has an on average 40% higher performance than Vanilla UCT.

Lastly, to test whether the performance improvements Global Std has over Vanilla UCT stem solely from Global Std being reward scale invariant, the pairings score for both the parameter-optimized and single-parameter setting was calculated when restricted to the two-player zero-sum games, which all have the same reward scale. The results are shown in Fig. 5 and Fig. 6 in the supplementary materials. Unsurprisingly, Vanilla UCT performs far better in this setting; however, in direct comparison, Global Std still has a notable lead over Vanilla UCT. This suggests that Global Std has effects beneficial to the performance that go beyond adapting the reward scale. We believe the global Q variance is a proxy for uncertainty and thus the amount of exploration required.

V. CONCLUSION AND FUTURE WORK

In this paper, we evaluated 12 different λ -strategies (5 from the literature and 7 from us) on a plethora of sequential-decision making problems with varying reward scales, showing that Vanilla UCT (i.e., using a fixed exploration constant) is not only highly scale dependent, even with a properly adjusted λ , Vanilla UCT is also outperformed by some of the other strategies. In particular, our newly proposed strategy Global Std (using $C = 2$), generalized best across the environments considered here, and also has the best overall parameter-optimized performance, decisively beating Vanilla UCT in both regards. Finally, we came to the conclusion that the Global Std strategy is a good, easy-to-implement replacement for Vanilla UCT.

For future work, we think that one can expand this work in two dimensions. Firstly, propose and evaluate more λ -strategies, for example, by taking a convex combination of the already introduced strategies or dynamically switching between strategies, as though Global Std performs best overall, it doesn’t do so for every single environment. Secondly, the evaluation may be conducted on a much larger suite of environments. For example, we did not include partially observable or deterministic single-agent environments, and we are far from exhausting the IPPC problem list.

REFERENCES

- [1] “What is Battlesnake?” docs.battlesnake.com/. Accessed: Jun. 3, 2025. [Online] Available: ‘https://docs.battlesnake.com/.
- [2] Ankit Anand, Ritesh Noothigattu, Mausam, and Parag Singla. OGA-UCT: on-the-go abstractions in UCT. In *Proceedings of the Twenty-Sixth International Conference on Automated Planning and Scheduling*, ICAPS’16, page 29–37. AAAI Press, 2016.
- [3] Jean-Yves Audibert, Rémi Munos, and Csaba Szepesvári. Use of variance estimation in the multi-armed bandit problem, 2006. NIPS Workshop on On-line Trading of Exploration and Exploitation Workshop.
- [4] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Mach. Learn.*, 47(2-3):235–256, 2002.
- [5] Radha-Krishna Balla and Alan Fern. UCT for Tactical Assault Planning in Real-Time Strategy Games. In Craig Boutilier, editor, *IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009*, pages 40–45, 2009.
- [6] Blai Bonet and Hector Geffner. Action Selection for MDPs: Anytime AO* Versus UCT. In Jörg Hoffmann and Bart Selman, editors, *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence, July 22-26, 2012, Toronto, Ontario, Canada*, pages 1749–1755. AAAI Press, 2012.
- [7] Cameron Browne, Edward Jack Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games*, 4(1):1–43, 2012.
- [8] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In H. Jaap van den Herik, Paolo Ciancarini, and H. H. L. M. Donkers, editors, *Computers and Games, 5th International Conference, CG 2006, Turin, Italy, May 29-31, 2006. Revised Papers*, volume 4630 of *Lecture Notes in Computer Science*, pages 72–83. Springer, 2006.
- [9] Alexander Dockhorn, Jorge Hurtado Grueso, Dominik Jeurissen, and Diego Perez Liebana. STRATEGA: A General Strategy Games Framework. In Joseph C. Osborn, editor, *Joint Proceedings of the AIIDE 2020 Workshops co-located with 16th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2020), Worcester, MA, USA, October 19-23, 2020 (online)*, volume 2862 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2020.
- [10] Tuomas Haarnoja et al. Soft actor-critic algorithms and applications. *CoRR*, abs/1812.05905, 2018.
- [11] Patrick Eyerich, Thomas Keller, and Malte Helmert. High-Quality Policies for the Canadian Traveler’s Problem. In Maria Fox and David Poole, editors, *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2010, Atlanta, Georgia, USA, July 11-15, 2010*, pages 51–58. AAAI Press, 2010.
- [12] Edgar Galván, Gavin Simpson, and Fred Valdez Ameneyro. Evolving the MCTS upper confidence bounds for trees using a semantic-inspired evolutionary algorithm in the game of carcassonne. *IEEE Trans. Games*, 15(3):420–429, 2023.
- [13] Robert C. Gray, Jichen Zhu, and Santiago Ontañón. Beyond uct: Mab exploration improvements for monte carlo tree search. In *2023 IEEE Conference on Games (CoG)*, 2023.
- [14] Marek Grzes, Jesse Hoey, and Scott Sanner. International Probabilistic Planning Competition (IPPC) 2014. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2014.
- [15] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
- [16] Levente Kocsis and Csaba Szepesvári. Bandit Based Monte-Carlo Planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006, 17th European Conference on Machine Learning, Berlin, Germany, September 18-22, 2006, Proceedings*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.
- [17] Tomáš Kozielek. Methods of MCTS and the game Arimaa. Master’s thesis, Charles University in Prague, Faculty of Mathematics and Physics, Prague, Czech Republic, 2009.
- [18] Diego Perez Liebana, Alexander Dockhorn, Jorge Hurtado Grueso, and Dominik Jeurissen. The Design Of “Stratega”: A General Strategy Games Framework. *CoRR*, abs/2009.05643, 2020.
- [19] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. The second type of uncertainty in monte carlo tree search. *CoRR*, abs/2005.09645, 2020.
- [20] Scott Sanner and Sungwook Yoon. International Probabilistic Planning Competition (IPPC) 2011. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2011.
- [21] Robin Schmöcker and Alexander Dockhorn. A survey of non-learning-based abstractions for sequential decision-making. *IEEE Access*, 13:100808–100830, 2025.
- [22] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy P. Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nat.*, 588(7839):604–609, 2020.
- [23] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences of the United States of America*, 39(10):1095–1100, October 1953.
- [24] Chiara F. Sironi, Jialin Liu, Diego Perez Liebana, Raluca D. Gaina, Ivan Bravi, Simon M. Lucas, and Mark H. M. Winands. Self-adaptive MCTS for General Video Game Playing. In Kevin Sim and Paul Kaufmann, editors, *Applications of Evolutionary Computation - 21st International Conference, EvoApplications 2018, Parma, Italy, April 4-6, 2018, Proceedings*, volume 10784 of *Lecture Notes in Computer Science*, pages 358–375. Springer, 2018.
- [25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.